

Stock Prediction with Mamba

A Quantative Investment Guide — From SSM Architecture to Trading Strategy

Contents

1	Introduction	3
2	Phase 1: Infrastructure	3
2.1	Matrix Class	3
2.2	Activation Functions	4
2.2.1	Sigmoid (Gate Control)	4
2.2.2	SiLU / Swish (Mamba Gating Activation)	4
2.2.3	Softplus (Δ Discretisation Activation)	4
2.2.4	tanh (Hidden State Activation)	5
2.2.5	Softmax (Classification Output)	5
2.2.6	Element-wise Exponential (SSM Discretisation)	5
3	Phase 2: Mamba Forward Pass	5
3.1	Why Mamba?	5
3.2	Network Parameters	6
3.3	Forward Equations	6
3.4	Caching for Backpropagation	8
4	Phase 3: Quant Loss Functions & Backpropagation	8
4.1	Mean Squared Error (MSE) for Return Prediction	8
4.2	Directional Accuracy Loss (Classification)	8
4.3	Sharpe Ratio Loss (Risk-Adjusted Return)	9
4.4	Pinball Loss (Quantile Regression for Value at Risk)	9
4.5	Composite Loss Function	10
4.6	Mamba Backpropagation (Core)	10
4.7	Gradient Clipping	12
5	Phase 4: Parameter Updates	12
5.1	Stochastic Gradient Descent (SGD)	12
5.2	Adam (Recommended Upgrade)	12
6	Phase 5: Weight Initialisation	13
6.1	Xavier Initialisation	13
6.2	Mamba-Specific Initialisation	13
7	Phase 6: Stock Data Preparation	14
7.1	Data Sources	14
7.2	Feature Engineering	14
7.3	Target Construction	14
7.4	Sequence Construction	15

8 Phase 7: Training Loop with Walk-Forward Validation	15
8.1 Complete Training Procedure	15
8.2 Hyperparameter Selection	15
8.3 Monitoring	15
9 Phase 8: Verification & Debugging	16
9.1 Numerical Gradient Checking	16
9.2 Common Bug Checklist	16
10 Phase 9: Advanced Directions	16
10.1 Multi-Asset Mamba	16
10.2 Attention-Free Portfolio Optimisation	17
10.3 Risk Decomposition with SSM States	17
10.4 Macro-Integration	17
10.5 Integrating Eigen for Performance	18
10.6 Parallel Scan (Associative Scan)	18
11 Phase 10: Three-Step Roadmap to Production Quant	18
11.1 Step 1: Train on Single Stock (1–2 weeks)	18
11.2 Step 2: Multi-Stock with Walk-Forward (1–2 months)	18
11.3 Step 3: GPU Training & Live Trading (requires hardware)	19
Appendix: Notation Reference	19

1 Introduction

Prerequisites: Multivariable calculus (partial derivatives, chain rule), linear algebra (matrix multiplication, vectors, transposes), and basic familiarity with financial time series (price, return, volatility).

Goal: Implement a trainable stock prediction system based on Mamba (Selective State Space Model) in C++ with LibTorch, from scratch. The model ingests financial features and outputs predicted returns, directional movements, or risk metrics for quantitative trading.

Why Mamba for Stock Prediction? Time series forecasting is the backbone of quantitative finance. Traditional approaches include:

- **ARIMA/GARCH:** Linear models that capture auto-regressive structure and volatility clustering, but cannot model complex non-linear dependencies in cross-sectional features.
- **LSTM/GRU:** Recurrent networks that handle non-linearity, but suffer from sequential training bottlenecks and vanishing gradient issues on long sequences.
- **Transformers:** Powerful for capturing long-range dependencies via attention, but quadratic $O(T^2)$ complexity in sequence length makes them expensive for high-frequency or long-context financial data.

Mamba (Selective State Space Model) overcomes these limitations:

- The linear recurrence $h_t = \bar{A} h_{t-1} + \bar{B} x_t$ can be computed in **parallel** via the associative scan algorithm — enabling efficient training on long financial histories.
- **Selectivity:** Making $\bar{A}$, $\bar{B}$, $\bar{C}$ input-dependent allows the model to dynamically adapt its memory: e.g., suppress noise during low-volatility periods and amplify signal at regime changes.
- Inference is $O(1)$ per step — critical for real-time trading systems where latency matters.

Roadmap:

1. Matrix infrastructure
2. Activation functions (sigmoid, SiLU, softplus, tanh, softmax)
3. Mamba forward pass
4. **Quant loss functions & backpropagation** (MSE, Sharpe ratio, directional accuracy, pinball loss)
5. Parameter updates (SGD, then Adam)
6. Weight initialisation
7. **Stock data preparation** (OHLCV, technical indicators, normalisation)
8. Training loop with walk-forward validation
9. Gradient checking & debugging
10. **Feature engineering** (from price series to predictive signals)
11. Advanced directions (multi-asset Mamba, attention-free portfolio optimisation, risk decomposition)
12. Three-step roadmap to production quant capability

2 Phase 1: Infrastructure

2.1 Matrix Class

All Mamba operations reduce to matrix algebra. Implement a **Matrix** struct with the following:

- **rows, cols:** dimensions.
- **data:** a flat `std::vector<double>` stored in row-major order, i.e. element (i, j) is at `data[i * cols + j]`.
- **at(i, j):** subscript access.

- **Matrix multiplication** $C = AB$:

$$C_{ij} = \sum_k A_{ik} B_{kj}$$

- **Transpose** $A^\top$:

$$(A^\top)_{ij} = A_{ji}$$

Used pervasively in backpropagation because $\frac{\partial(Wx)}{\partial x} = W^\top$.

- **Element-wise addition** $(A + B)_{ij} = A_{ij} + B_{ij}$ (same shape required).
- **Hadamard product** (element-wise multiplication) $(A \odot B)_{ij} = A_{ij} \cdot B_{ij}$, used for gradient gating in backprop.

Why transpose? In the forward pass, a linear layer computes $y = Wx$. During backpropagation, the gradient flows as $\frac{\partial L}{\partial x} = W^\top \frac{\partial L}{\partial y}$. The transpose is not an arbitrary convention—it is the mathematical consequence of the multivariable chain rule applied to matrix differentiation.

2.2 Activation Functions

Implement each function *and* its derivative (the derivative is what backpropagation needs).

2.2.1 Sigmoid (Gate Control)

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

Derivation. Let $s = \sigma(x)$. Then $\frac{ds}{dx} = s(1 - s)$.

Numerical stability. For $x \geq 0$, use $\frac{1}{1+e^{-x}}$; for $x < 0$, use $\frac{e^x}{1+e^x}$. This avoids overflow in either direction.

2.2.2 SiLU / Swish (Mamba Gating Activation)

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

$$\text{SiLU}'(x) = \sigma(x) + x \cdot \sigma(x)(1 - \sigma(x)) = \sigma(x)(1 + x - x\sigma(x))$$

Derivation. Let $s = \sigma(x)$. Then $\frac{d}{dx}[x \cdot s] = s + x \cdot s(1 - s) = s(1 + x - xs)$.

Why SiLU for Mamba? SiLU replaces the sigmoid gating of LSTM. It combines linear and non-linear characteristics: in the negative region it has a soft suppression effect, and its gradient is smoother than a plain sigmoid, which helps training stability.

2.2.3 Softplus (Δ Discretisation Activation)

$$\text{softplus}(x) = \log(1 + e^x)$$

Numerical stability. For $x > 20$, return x directly; for $x < -20$, return e^x ; otherwise use $\log1p(\exp(x))$.

Why softplus for Δ ? Softplus is a smooth approximation of ReLU that is always positive. It ensures the discretisation step size Δ remains positive, which is essential for the SSM discretisation to be well-defined.

Derivative.

$$\text{softplus}'(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

2.2.4 tanh (Hidden State Activation)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\tanh'(x) = 1 - \tanh^2(x)$$

Derivation. Let $t = \tanh(x)$. Then $\frac{dt}{dx} = 1 - t^2$.

2.2.5 Softmax (Classification Output)

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Numerical stability: Subtract the row maximum before exponentiating:

$$\text{softmax}(z)_i = \frac{e^{z_i - \max_k z_k}}{\sum_j e^{z_j - \max_k z_k}}$$

Softmax + cross-entropy joint gradient (for directional classification).

$$\frac{\partial}{\partial z_i} (-\log p_y) = p_i - \mathbf{1}_{i=y}$$

This closed-form gradient is used when the output is a ternary direction (up / down / neutral). See Section 4.2.

2.2.6 Element-wise Exponential (SSM Discretisation)

$$\exp(A)_{ij} = e^{A_{ij}}$$

Used in SSM discretisation to compute $\bar{A} = \exp(\Delta \odot A)$. No special derivative formula is needed beyond $\frac{d}{dx} e^x = e^x$; the element-wise exponential is applied to stored matrices during the forward pass and its result is cached for backpropagation.

3 Phase 2: Mamba Forward Pass

3.1 Why Mamba?

LSTM mitigates vanishing gradients through gating and additive cell-state updates, but still has limitations:

1. **Cannot parallelise training:** The sequential dependency $h_t = f(h_{t-1}, x_t)$ requires strictly sequential computation; the sequence dimension cannot be parallelised.
2. **Long-range dependencies still limited:** Although better than vanilla RNNs, gradients must still traverse every time step.
3. **Inefficient inference:** Each time step depends on the previous one; no batch processing is possible.

Mamba's core innovation is *selective scan*:

- The linear recurrence of state space models, $h_t = \bar{A}h_{t-1} + \bar{B}x_t$, can be computed in parallel via the associative scan algorithm.
- **Selectivity:** Making $\bar{A}$, $\bar{B}$, $\bar{C}$ input-dependent allows the model to selectively propagate or forget information based on content.
- Inference is $O(1)$ state update per step (no need to cache the entire history as Transformers do).

3.2 Network Parameters

Symbol	Name	Shape	Role
W_{in}	Input projection weight	$(2 d_{\text{model}}) \times d_x$	Project input features to model dimension; $2\times$ for gate
W_{conv}	Conv1d kernel	$d_{\text{model}} \times d_{\text{conv}}$	Causal 1D convolution kernel
b_{conv}	Conv1d bias	$d_{\text{model}} \times 1$	Convolution bias
A_{log}	SSM diagonal matrix (log)	$d_{\text{model}} \times 1$	Log of diagonal A elements
D	Skip connection	$d_{\text{model}} \times 1$	Residual connection parameter
W_{Δ}	Delta projection weight	$d_{\text{model}} \times d_{\text{model}}$	Input-dependent discretisation step
b_{Δ}	Delta bias	$d_{\text{model}} \times 1$	Step size bias
W_B	B projection weight	$d_{\text{state}} \times d_{\text{model}}$	Input-dependent B matrix
W_C	C projection weight	$d_{\text{state}} \times d_{\text{model}}$	Input-dependent C matrix
W_{out}	Output projection weight	$d_y \times d_{\text{model}}$	Map hidden state to prediction space

Table 1: Mamba parameters for stock prediction. d_x = number of input features, d_{model} = hidden dimension, d_{state} = SSM state dimension (typically 16), d_{conv} = convolution kernel size (typically 3), d_y = output dimension (1 for return regression, 3 for ternary direction).

3.3 Forward Equations

Given input sequence $\{x_1, x_2, \dots, x_T\}$ where each $x_t \in \mathbb{R}^{d_x}$ is a vector of stock features at time t , initial state $h_0 = \mathbf{0}$.

Step 1: Input projection.

$$z = W_{\text{in}} x_t \quad \text{shape: } (2 d_{\text{model}}) \times 1 \quad (1)$$

Split z into two halves:

$$z_1 = z[0 : d_{\text{model}}] \quad (\text{SSM branch}) \quad (2)$$

$$z_2 = z[d_{\text{model}} : 2 d_{\text{model}}] \quad (\text{gate branch}) \quad (3)$$

z_1 enters the SSM branch; z_2 is used for gating.

Step 2: Causal convolution. Apply causal 1D convolution (kernel size d_{conv}) to the z_1 sequence:

$$x'_{\text{conv},t} = \sum_{k=0}^{d_{\text{conv}}-1} W_{\text{conv}}[:,k] \cdot z_1[t-k] + b_{\text{conv}} \quad (4)$$

where $z_1[t-k]$ is taken as zero when $t-k < 0$ (causal padding).

Step 3: Selective parameter computation (S6 core). Compute input-dependent parameters from the convolution output $x'_{\text{conv},t}$:

$$\Delta_t = \text{softplus}(W_\Delta x'_{\text{conv},t} + b_\Delta) \quad \text{step size, shape: } d_{\text{model}} \times 1 \quad (5)$$

$$B_t = W_B x'_{\text{conv},t} \quad \text{input matrix, shape: } d_{\text{state}} \times 1 \quad (6)$$

$$C_t = W_C x'_{\text{conv},t} \quad \text{output matrix, shape: } d_{\text{state}} \times 1 \quad (7)$$

Step 4: SSM discretisation. Discretise the continuous parameters:

$$\bar{A}_t = \exp(\Delta_t \odot A) \quad \text{shape: } d_{\text{model}} \times 1 \quad (8)$$

$$\bar{B}_t = \Delta_t \odot B_t^\top \quad \text{shape: } d_{\text{model}} \times d_{\text{state}} \quad (9)$$

where $A = \exp(A_{\log})$ recovers the diagonal matrix elements from log-space.

Step 5: Selective scan (recurrence). For each channel i (there are d_{model} channels), each channel has an independent state $h_t[i] \in \mathbb{R}^{d_{\text{state}}}$:

$$h_t[i] = \bar{A}_t[i] \cdot h_{t-1}[i] + \bar{B}_t[i, :] x'_{\text{conv},t}[i] \quad (10)$$

$$y_{\text{ssm},t}[i] = C_t^\top h_t[i] + D[i] \cdot x'_{\text{conv},t}[i] \quad (11)$$

Simplified implementation. In the matrix framework, store h_t as a $d_{\text{model}} \times d_{\text{state}}$ matrix, $\bar{A}_t$ as a $d_{\text{model}} \times 1$ vector, and the recurrence becomes:

$$h_t = \bar{A}_t \odot h_{t-1} + \bar{B}_t \odot x'_{\text{conv},t} \quad (\text{broadcast } \odot) \quad (12)$$

$$y_{\text{ssm},t} = C_t^\top h_t + D \odot x'_{\text{conv},t} \quad (13)$$

(Here $\odot$ denotes broadcast multiplication: $\bar{A}_t$ scales each row of h_{t-1} , and $\bar{B}_t$ scales $x'_{\text{conv},t}$ across the state dimension.)

Step 6: Gating.

$$y_t = \text{SiLU}(z_2) \odot y_{\text{ssm},t} \quad (14)$$

Step 7: Output projection. For **return regression** (single output):

$$\hat{r}_t = W_{\text{out}} y_t \quad \text{shape: } 1 \times 1 \quad (15)$$

For **directional classification** (ternary: up / down / neutral):

$$\hat{p}_t = \text{softmax}(W_{\text{out}} y_t) \quad \text{shape: } 3 \times 1 \quad (16)$$

Financial intuition of the selective mechanism.

- Δ_t : controls how quickly the model adapts to new market information. Large Δ = rapid adaptation (e.g., after news events); small Δ = slow, smooth filtering (e.g., during low-volatility periods).
- B_t : input selection — decides which market signals are allowed into the latent state. A low-B regime suppresses noisy features; high-B regime amplifies predictive signals.
- C_t : output selection — controls which parts of the learned market state are read for prediction. The model can learn to focus on trend-following components in bull markets and mean-reversion components in range-bound markets.
- $\bar{A}_t = \exp(\Delta_t \odot A)$: state decay. In financial terms, this is the “memory half-life” of the model. A $\bar{A}$ near 1 means long memory (trend regime); near 0 means short memory (mean-reversion regime).

3.4 Caching for Backpropagation

Backpropagation requires intermediate values from the forward pass. Store the following at every time step:

Cache variable	Formula role	Backprop usage
x_t	Current input features	Compute weight gradients
z	Projected input	Split into z_1, z_2
z_1	SSM branch input	Compute convolution and SSM gradients
z_2	Gate branch input	Compute SiLU gradient
$x'_{\text{conv},t}$	Convolution output	Compute Δ, B, C and SSM gradients
Δ_t	Discretised step size	Compute $\bar{A}, \bar{B}$ gradients
B_t	Input matrix	Compute SSM gradients
C_t	Output matrix	Compute SSM gradients
h_t	SSM hidden state	Compute recurrence gradients
$y_{\text{ssm},t}$	SSM output	Compute gating gradient
y_t	Gated output	Compute output projection gradient

Design suggestion: Define a `MambaCache` struct containing `std::vector<Matrix>` members for the quantities above.

4 Phase 3: Quant Loss Functions & Backpropagation

This section is the heart of the transition from NLP to quantitative finance. Instead of cross-entropy on character tokens, we define loss functions that directly optimise trading and risk metrics.

4.1 Mean Squared Error (MSE) for Return Prediction

The simplest quant loss: predict the next-period return r_t and minimise squared error.

$$L_{\text{MSE}} = \frac{1}{T} \sum_{t=1}^T (r_t - \hat{r}_t)^2$$

where $\hat{r}_t = W_{\text{out}} y_t$ is the predicted return at step t , and r_t is the actual return.

Gradient.

$$\frac{\partial L_{\text{MSE}}}{\partial \hat{r}_t} = -\frac{2}{T} (r_t - \hat{r}_t)$$

Output layer gradient (regression case).

$$\delta_t^{\text{out}} = \frac{\partial L_{\text{MSE}}}{\partial o_t^{\text{out}}} = \frac{\partial L_{\text{MSE}}}{\partial \hat{r}_t} \cdot \frac{\partial \hat{r}_t}{\partial o_t^{\text{out}}} = -\frac{2}{T} (r_t - \hat{r}_t)$$

where $o_t^{\text{out}} = W_{\text{out}} y_t$ is the output logit.

4.2 Directional Accuracy Loss (Classification)

For directional trading, predict three classes: UP (price up), DOWN (price down), NEUTRAL (price unchanged within threshold). Use cross-entropy loss:

$$L_{\text{Dir}} = -\frac{1}{T} \sum_{t=1}^T \log \hat{p}_t[y_t]$$

where $y_t \in \{0, 1, 2\}$ is the true direction and $\hat{p}_t$ is the softmax output.

Output layer gradient. The joint gradient of softmax + cross-entropy has the same closed form:

$$\frac{\partial L_{\text{Dir}}}{\partial o_t^{\text{out}}} = \hat{p}_t - \mathbf{1}_{y_t}$$

4.3 Sharpe Ratio Loss (Risk-Adjusted Return)

The Sharpe ratio measures excess return per unit of risk. We define a differentiable Sharpe ratio loss that optimises the entire prediction sequence jointly.

Given predicted returns $\{\hat{r}_1, \dots, \hat{r}_T\}$, the realised Sharpe ratio (assuming zero risk-free rate) is:

$$\text{SR} = \frac{\mu(\hat{r})}{\sigma(\hat{r})} \cdot \sqrt{252}$$

where $\mu(\hat{r}) = \frac{1}{T} \sum_{t=1}^T \hat{r}_t$ and $\sigma(\hat{r}) = \sqrt{\frac{1}{T-1} \sum_{t=1}^T (\hat{r}_t - \mu)^2}$.

The loss to minimise is:

$$L_{\text{SR}} = -\text{SR}$$

Gradient derivation. Let $\mu = \frac{1}{T} \sum_t \hat{r}_t$ and $\sigma^2 = \frac{1}{T-1} \sum_t (\hat{r}_t - \mu)^2$. Then:

$$\frac{\partial L_{\text{SR}}}{\partial \hat{r}_t} = -\frac{\sqrt{252}}{T} \left(\frac{1}{\sigma} - \frac{\mu}{\sigma^3} (\hat{r}_t - \mu) \right)$$

Key insight for BPTT. Unlike MSE where δ_t^{out} depends only on the current prediction, the Sharpe ratio gradient at time t depends on *all* predictions through μ and σ . This means the gradient flows through the entire sequence, making the backpropagation through the SSM inherently global. The Mamba's selective scan must propagate this full-sequence gradient.

Output layer gradient.

$$\delta_t^{\text{out}} = \frac{\partial L_{\text{SR}}}{\partial o_t^{\text{out}}} = \frac{\partial L_{\text{SR}}}{\partial \hat{r}_t}$$

4.4 Pinball Loss (Quantile Regression for Value at Risk)

For risk management, predict quantiles of the return distribution. The pinball loss for quantile $\tau \in (0, 1)$ is:

$$L_{\tau} = \frac{1}{T} \sum_{t=1}^T \rho_{\tau}(r_t - \hat{q}_t^{(\tau)})$$

where the pinball function is:

$$\rho_{\tau}(u) = u \cdot (\tau - \mathbb{1}_{u < 0})$$

Common choices: $\tau = 0.05$ (VaR 5%), $\tau = 0.01$ (VaR 1%), and $\tau = 0.5$ (median prediction).

Gradient.

$$\frac{\partial L_\tau}{\partial \hat{q}_t^{(\tau)}} = \frac{1}{T} (\mathbb{1}_{r_t > \hat{q}_t^{(\tau)}} \cdot \tau + \mathbb{1}_{r_t \leq \hat{q}_t^{(\tau)}} \cdot (\tau - 1))$$

Interpretation. The pinball loss is asymmetric: underestimating VaR (predicting too low a loss when the actual loss is larger) is penalised more heavily when $\tau < 0.5$. This matches the risk-averse nature of VaR.

4.5 Composite Loss Function

For real-world quant trading, a single loss function is rarely sufficient. We define a composite loss:

$$L = \lambda_1 L_{\text{MSE}} + \lambda_2 L_{\text{SR}} + \lambda_3 L_{\text{Dir}} + \lambda_4 L_{\text{Pinball}} + \lambda_5 L_{\text{Reg}}$$

where:

- L_{MSE} ensures basic predictive accuracy.
- L_{SR} optimises risk-adjusted returns.
- L_{Dir} improves direction prediction (trading signal quality).
- L_{Pinball} controls tail risk (can be multi-quantile: $\tau = 0.05, 0.25, 0.5, 0.75, 0.95$).
- $L_{\text{Reg}} = \frac{\gamma}{2} \sum_i \theta_i^2$ is weight decay regularisation.

The total gradient is the weighted sum:

$$\frac{\partial L}{\partial \hat{r}_t} = \lambda_1 \frac{\partial L_{\text{MSE}}}{\partial \hat{r}_t} + \lambda_2 \frac{\partial L_{\text{SR}}}{\partial \hat{r}_t} + \lambda_3 \frac{\partial L_{\text{Dir}}}{\partial \hat{r}_t} + \lambda_4 \frac{\partial L_{\text{Pinball}}}{\partial \hat{r}_t}$$

4.6 Mamba Backpropagation (Core)

This is the heart of the project. Mamba backpropagation is more complex than LSTM because gradients flow through the *selective scan recurrence path* and the *input-dependent parameterisation path* simultaneously.

Notation. Define the following gradient symbols:

$$\begin{aligned} \delta_t^{\text{out}} &= \frac{\partial L}{\partial o_t^{\text{out}}} && \text{(output layer gradient)} \\ \delta_t^y &= \frac{\partial L}{\partial y_t} && \text{(gated output gradient)} \\ \delta_t^{\text{ssm}} &= \frac{\partial L}{\partial y_{\text{ssm},t}} && \text{(SSM output gradient)} \\ \delta_t^h &= \frac{\partial L}{\partial h_t} && \text{(hidden state gradient)} \end{aligned}$$

The key difference from the NLP case: δ_t^{out} now comes from the quant loss functions above (Sharpe, MSE, pinball, etc.) rather than from cross-entropy on next-token prediction.

Step 1: Output layer to gated output.

$$\frac{\partial L}{\partial y_t} = W_{\text{out}}^\top \delta_t^{\text{out}}$$

Step 2: Gated output to SSM output and gate branch. From $y_t = \text{SiLU}(z_2) \odot y_{\text{ssm},t}$:

$$\frac{\partial L}{\partial y_{\text{ssm},t}} = \frac{\partial L}{\partial y_t} \odot \text{SiLU}(z_2) \quad (17)$$

$$\frac{\partial L}{\partial z_2} = \frac{\partial L}{\partial y_t} \odot y_{\text{ssm},t} \odot \text{SiLU}'(z_2) \quad (18)$$

Step 3: SSM output to hidden state. From $y_{\text{ssm},t} = C_t^\top h_t + D \odot x'_{\text{conv},t}$:

$$\frac{\partial L}{\partial h_t} += \delta_t^{\text{ssm}} \cdot C_t \quad (19)$$

$$\frac{\partial L}{\partial C_t} = h_t \cdot (\delta_t^{\text{ssm}})^\top \quad (20)$$

$$\frac{\partial L}{\partial D} += \delta_t^{\text{ssm}} \odot x'_{\text{conv},t} \quad (21)$$

Step 4: Through the selective scan recurrence. From $h_t = \bar{A}_t \odot h_{t-1} + \bar{B}_t \odot x'_{\text{conv},t}$:

$$\frac{\partial L}{\partial h_{t-1}} += \frac{\partial L}{\partial h_t} \odot \bar{A}_t \quad (22)$$

$$\frac{\partial L}{\partial \bar{A}_t} = \frac{\partial L}{\partial h_t} \odot h_{t-1} \quad (23)$$

$$\frac{\partial L}{\partial \bar{B}_t} = \frac{\partial L}{\partial h_t} \odot x'_{\text{conv},t} \quad (24)$$

Key insight for quant finance. Equation (22) is the gradient flow of the state h_t . In a vanilla RNN, $\frac{\partial h_t}{\partial h_{t-1}}$ involves a full matrix multiplication W_{hh} , causing exponential decay or explosion. In Mamba, $\frac{\partial h_t}{\partial h_{t-1}} = \bar{A}_t$ is an *element-wise* product. Since $\bar{A}_t \in (0, 1)$ by construction, gradients of early time steps decay smoothly — critical for learning long-term dependencies in financial data (e.g., multi-year mean reversion or trend signals). Moreover, the input-dependent $\bar{A}_t$ lets the model learn when to retain vs. forget market regimes.

Step 5: From discretised to continuous parameters. From $\bar{A}_t = \exp(\Delta_t \odot A)$ and $\bar{B}_t = \Delta_t \odot B_t^\top$:

$$\frac{\partial L}{\partial \Delta_t} = \frac{\partial L}{\partial \bar{A}_t} \odot \bar{A}_t \odot A + \frac{\partial L}{\partial \bar{B}_t} \odot B_t^\top \quad (25)$$

$$\frac{\partial L}{\partial A} += \sum_t \frac{\partial L}{\partial \bar{A}_t} \odot \bar{A}_t \odot \Delta_t \quad (26)$$

$$\frac{\partial L}{\partial B_t} = \Delta_t \cdot \left(\frac{\partial L}{\partial \bar{B}_t} \right)^\top \quad (27)$$

Step 6: From Δ , B , C to convolution output. From $\Delta_t = \text{softplus}(W_\Delta x'_{\text{conv},t} + b_\Delta)$:

$$\frac{\partial L}{\partial x'_{\text{conv},t}} += W_\Delta^\top \cdot \left(\frac{\partial L}{\partial \Delta_t} \odot \sigma(W_\Delta x'_{\text{conv},t} + b_\Delta) \right) + W_B^\top \cdot \frac{\partial L}{\partial B_t} + W_C^\top \cdot \frac{\partial L}{\partial C_t} \quad (28)$$

Step 7: From convolution output to input projection. Backpropagation through the causal convolution requires handling the causal padding and kernel reversal. The gradient then flows through z_1 back to the input projection.

Step 8: Parameter gradients (accumulated over all time steps).

$$\frac{\partial L}{\partial W_{\text{in}}} = \sum_{t=1}^T \frac{\partial L}{\partial z} x_t^\top \quad (29)$$

$$\frac{\partial L}{\partial W_{\text{out}}} = \sum_{t=1}^T \delta_t^{\text{out}} y_t^\top \quad (30)$$

$$\frac{\partial L}{\partial W_{\Delta}} = \sum_{t=1}^T \frac{\partial L}{\partial \Delta_t} \odot \sigma'(\cdot) (x'_{\text{conv},t})^\top \quad (31)$$

$$\frac{\partial L}{\partial W_B} = \sum_{t=1}^T \frac{\partial L}{\partial B_t} (x'_{\text{conv},t})^\top \quad (32)$$

$$\frac{\partial L}{\partial W_C} = \sum_{t=1}^T \frac{\partial L}{\partial C_t} (x'_{\text{conv},t})^\top \quad (33)$$

Bias gradients are analogous:

$$\frac{\partial L}{\partial b_{\text{conv}}} = \sum_{t=1}^T \frac{\partial L}{\partial x'_{\text{conv},t}}, \quad \frac{\partial L}{\partial b_{\Delta}} = \sum_{t=1}^T \frac{\partial L}{\partial \Delta_t} \odot \sigma'(\cdot) \quad (34)$$

The gradient for $A_{\log}$ is obtained via the chain rule through $\frac{\partial L}{\partial A}$ (Equation (26)) and the relation $A = \exp(A_{\log})$:

$$\frac{\partial L}{\partial A_{\log}} = \frac{\partial L}{\partial A} \odot A$$

4.7 Gradient Clipping

Although Mamba's selective scan mitigates vanishing gradients by design, exploding gradients can still occur (especially with poor $A_{\log}$ initialisation or excessive Δ). Clip each element of every gradient matrix:

$$\text{grad}_i \leftarrow \begin{cases} \text{grad}_i & \text{if } |\text{grad}_i| \leq c \\ c \cdot \text{sign}(\text{grad}_i) & \text{otherwise} \end{cases}$$

where c is a threshold (typical value: 5).

5 Phase 4: Parameter Updates

5.1 Stochastic Gradient Descent (SGD)

The simplest optimiser:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} L$$

where η is the learning rate. Implement this first to validate the pipeline.

5.2 Adam (Recommended Upgrade)

Once SGD works, upgrade to Adam:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (35)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (36)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (37)$$

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (38)$$

Symbol	Meaning
m_t	First moment estimate (exponential moving average of gradients)—direction
v_t	Second moment estimate (exponential moving average of squared gradients)—scale
$\hat{m}_t, \hat{v}_t$	Bias-corrected estimates (compensate for zero-initialisation)
β_1, β_2	Decay rates; typical values: 0.9 and 0.999
ϵ	Numerical stabiliser; typical value: 10^{-8}

Table 2: Adam optimiser notation.

Why Adam is better. Adam adapts the learning rate per-parameter. Mamba has multiple parameter groups (A , Δ , B , C , etc.) with drastically different gradient scales, making adaptive learning rates especially important.

6 Phase 5: Weight Initialisation

Never initialise all weights to zero! Zero initialisation makes all neurons learn identical features (symmetry problem).

6.1 Xavier Initialisation

$$W_{ij} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{\text{in}} + n_{\text{out}}}}\right)$$

where n_{in} is the input dimension and n_{out} is the output dimension of the layer.

Theory. The goal is to keep the variance of activations stable across layers. If the input x has variance σ_x^2 and weights have variance σ_W^2 :

$$\text{Var}(Wx) = n_{\text{in}} \cdot \sigma_W^2 \cdot \sigma_x^2$$

For $\text{Var}(Wx) = \sigma_x^2$, we need $\sigma_W^2 = \frac{1}{n_{\text{in}}}$. Considering also backpropagation stability, we take the harmonic mean $\frac{2}{n_{\text{in}} + n_{\text{out}}}$.

6.2 Mamba-Specific Initialisation

- **$A_{\log}$ initialised to negative values:** Set $A_{\log,i} = -1$ (i.e. $A_i = e^{-1} \approx 0.368$). This ensures $\bar{A} \in (0, 1)$ so the state decays naturally.
- **D initialised to zero:** The skip connection starts at zero, forcing the SSM branch to learn first.
- **b_{Δ} initialised to a small positive value** (e.g. 0.001): Ensures the initial step size Δ is positive and small.
- All other weights use Xavier initialisation.

7 Phase 6: Stock Data Preparation

7.1 Data Sources

The primary data for stock prediction is:

- **OHLCV:** Open, High, Low, Close, Volume — daily or intraday.
- **Adjusted prices:** Accounting for splits and dividends for long-term consistency.
- **Fundamental data (optional):** P/E ratio, earnings, book value, etc. (lower frequency).

7.2 Feature Engineering

Raw prices are non-stationary; they must be transformed:

Returns (stationary).

$$r_t = \frac{p_t - p_{t-1}}{p_{t-1}} \quad (\text{simple return}), \quad \tilde{r}_t = \log\left(\frac{p_t}{p_{t-1}}\right) \quad (\text{log return})$$

Normalisation (z-score). Each feature is normalised to zero mean and unit variance using a rolling window:

$$\tilde{x}_t^{(i)} = \frac{x_t^{(i)} - \mu_t^{(i)}}{\sigma_t^{(i)}}$$

where μ_t and σ_t are estimated from a trailing window (e.g., 252 trading days). This prevents look-ahead bias.

Technical Indicators (common predictive features).

$$\text{SMA}_n(t) = \frac{1}{n} \sum_{k=0}^{n-1} p_{t-k} \quad (\text{Simple Moving Average})$$

$$\text{RSI}(t) = 100 - \frac{100}{1 + \frac{\text{avg gain}_n(t)}{\text{avg loss}_n(t)}} \quad (\text{Relative Strength Index})$$

$$\text{Bollinger}(t) = \frac{p_t - \text{SMA}_n(t)}{\sigma_n(t)} \quad (\text{Bollinger Band position})$$

$$\text{Volume ratio}(t) = \frac{V_t}{\text{SMA}_n^V(t)} \quad (\text{Relative volume})$$

$$\text{Volatility}(t) = \sqrt{\frac{1}{n-1} \sum_{k=0}^{n-1} (r_{t-k} - \bar{r})^2} \quad (\text{Realised volatility})$$

The full feature vector $x_t \in \mathbb{R}^{d_x}$ concatenates these indicators for a given stock.

7.3 Target Construction

The prediction target depends on the loss function:

- **Regression:** Next-period return $r_{t+1} = \frac{p_{t+1} - p_t}{p_t}$.
- **Classification:** Direction $y_t = \begin{cases} 0 & \text{if } r_{t+1} < -\epsilon \\ 1 & \text{if } |r_{t+1}| \leq \epsilon \quad (\text{ternary: down / neutral / up}). \\ 2 & \text{if } r_{t+1} > \epsilon \end{cases}$
- **Quantile regression:** Future return quantile $\hat{q}_t^{(\tau)}$ at level τ .

7.4 Sequence Construction

Given N trading days of feature vectors $\{x_1, \dots, x_N\}$:

- Slide a window of length T : input $x_{t-T+1}, \dots, x_t$, target y_{t+1} (next step).
- **Critical:** Never use future data to construct features for the current step (no look-ahead bias). All normalisation and indicator computations must use only data available at time t .
- Train/validation/test split must respect temporal order: first 70% time, next 15%, last 15%.

8 Phase 7: Training Loop with Walk-Forward Validation

8.1 Complete Training Procedure

Unlike NLP (where data can be randomly shuffled), financial time series require temporal integrity.

1. For each epoch (pass over training period):
 - (a) For each mini-batch (consecutive sub-sequence of length T):
 - i. **Forward pass:** compute $z, z_1, z_2, x'_{\text{conv}}, \Delta, B, C, h, y_{\text{ssm}}, y, \hat{r}_t$ (or $\hat{p}_t$) for all time steps.
 - ii. **Compute loss:** $L = \lambda_1 L_{\text{MSE}} + \lambda_2 L_{\text{SR}} + \lambda_3 L_{\text{Dir}} + \lambda_4 L_{\text{Pinball}} + \lambda_5 L_{\text{Reg}}$.
 - iii. **Backward pass (BPTT):** propagate gradients from step T back to step 1 through the SSM recurrence.
 - iv. **Gradient clipping:** $|\text{grad}_i| = \min(|\text{grad}_i|, 5)$.
 - v. **Parameter update:** $\theta \leftarrow \theta - \eta \nabla_{\theta} L$ (or Adam update).
 - (b) Log the average loss and validation Sharpe ratio for this epoch.
2. **Walk-forward validation:** After each epoch, roll the training window forward by K days and re-train. The model is evaluated on out-of-sample data that follows the training window.

Walk-forward schematic.

$$\underbrace{\text{Train}_{[t_0, t_1]}}_{\text{epoch 1}} \rightarrow \underbrace{\text{Val}_{[t_1, t_2]}}_{\text{epoch 1 eval}} \rightarrow \underbrace{\text{Train}_{[t_0+K, t_1+K]}}_{\text{epoch 2}} \rightarrow \underbrace{\text{Val}_{[t_1+K, t_2+K]}}_{\text{epoch 2 eval}} \rightarrow \dots$$

8.2 Hyperparameter Selection

8.3 Monitoring

- **Training loss:** should decrease steadily.
- **Validation Sharpe ratio:** the primary metric. Declining Sharpe $\rightarrow$ overfitting.
- **Directional accuracy:** percentage of correctly predicted price directions.
- **Gradient norm:** $\|\nabla\|_2 = \sqrt{\sum_i g_i^2}$. Consistently growing $\rightarrow$ gradient explosion.
- **Prediction bias:** $\frac{1}{T} \sum_t \hat{r}_t$. Large positive/negative bias $\rightarrow$ model is not calibrated.
- **Δ mean:** monitor Δ_t . Near zero $\rightarrow$ model is “freezing” the state; very large $\rightarrow$ model is “skipping” information (unstable).
- **Regime detection:** plot $\bar{A}_t$ over time. Regime changes should correspond to shifts in $\bar{A}_t$ (high memory $\rightarrow$ low memory transitions).

Hyperparameter	Suggested initial value	Notes
Feature dimension d_x	20–50	Number of technical indicators + returns
Model dimension d_{model}	64–128	Too small $\rightarrow$ underfitting; too large $\rightarrow$ overfitting + slow
State dimension d_{state}	16	SSM state dimension; typically 16
Conv kernel size d_{conv}	3	Causal convolution kernel size
Sequence length T	30–120	Trading days. 63 = one quarter; 252 = one year
Learning rate η	0.001 (Adam) / 0.1 (SGD)	Adam needs a smaller learning rate
Loss weights $\lambda_1 : \lambda_2 : \lambda_3 : \lambda_4 : \lambda_5$	1:0.5:1:0.1:0.01	Tune based on Sharpe vs. accuracy trade-off
Clip threshold	5	Empirical; adjust based on gradient norm
Epochs	50–200	Monitor validation Sharpe; early-stop when it declines

9 Phase 8: Verification & Debugging

9.1 Numerical Gradient Checking

After implementing backpropagation, **always** verify with numerical gradients:

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta + \epsilon \mathbf{e}_i) - L(\theta - \epsilon \mathbf{e}_i)}{2\epsilon}$$

with $\epsilon \approx 10^{-4}$.

Acceptance criterion.

$$\frac{\|\mathbf{g}_{\text{analytic}} - \mathbf{g}_{\text{numeric}}\|_2}{\max(\|\mathbf{g}_{\text{analytic}}\|_2, \|\mathbf{g}_{\text{numeric}}\|_2)} < 10^{-5}$$

Mamba-specific note. Mamba has 10 parameter matrices ($W_{\text{in}}, W_{\text{conv}}, b_{\text{conv}}, A_{\text{log}}, D, W_{\Delta}, b_{\Delta}, W_B, W_C, W_{\text{out}}$)—verify every single one. For quant loss functions, verify each loss component separately before testing the composite.

9.2 Common Bug Checklist

10 Phase 9: Advanced Directions

10.1 Multi-Asset Mamba

Extend the model to predict returns for multiple assets simultaneously:

$$\mathbf{r}_t = (r_t^{(1)}, r_t^{(2)}, \dots, r_t^{(N)})^\top, \quad \hat{\mathbf{r}}_t = W_{\text{out}} y_t$$

with $W_{\text{out}} \in \mathbb{R}^{N \times d_{\text{model}}}$. The loss becomes a portfolio-level objective:

$$L_{\text{portfolio}} = -\text{SR}_{\text{portfolio}} + \lambda \cdot \text{Turnover penalty}$$

where $\text{SR}_{\text{portfolio}}$ is the Sharpe ratio of a portfolio constructed from the N asset predictions.

Symptom	Possible cause
Loss not decreasing	Learning rate too large or too small; gradient sign error
Loss becomes NaN	Numerical overflow; Δ went negative (softplus should prevent this); exp overflow
Sharpe ratio not improving	MSE and Sharpe gradients in conflict; loss weights λ_i poorly balanced
All gradients zero	Forgot to accumulate across time steps; initialisation issue
Gradient explosion	No gradient clipping; Δ too large causing $\bar{A}$ overflow
Sharpe ratio NaN at start	First epoch predictions are near-zero; $\sigma(\hat{r}) \approx 0$ causes division by zero. Add ϵ to denominator: $\sigma(\hat{r}) + 10^{-8}$
Numerical check fails	Missing or wrong-sign $\frac{\partial}{\partial(\cdot)}$ term in back-prop; or quant loss gradient not properly chained
Δ all zeros	b_Δ initialised improperly; softplus implementation error
State h_t explosion	A initialised to positive values causing $\bar{A} > 1$; ensure $A_{\log} < 0$
Model always predicts zero return	Bias terms missing or W_{out} poorly initialised
Overfitting (high train Sharpe, negative test Sharpe)	Too many parameters for the amount of data; increase L_{Reg} weight or reduce d_{model}

10.2 Attention-Free Portfolio Optimisation

Mamba's $O(T)$ complexity and $O(1)$ per-step inference make it ideal for real-time portfolio optimisation. Replace traditional quadratic programming (Markowitz) with a learned Mamba policy that outputs portfolio weights directly:

$$w_t = \text{softmax}(W_{\text{portfolio}} y_t)$$

where $w_t \in \mathbb{R}^N$ are asset weights. Train end-to-end with a differentiable portfolio loss.

10.3 Risk Decomposition with SSM States

The SSM hidden state $h_t \in \mathbb{R}^{d_{\text{model}} \times d_{\text{state}}}$ naturally factorises market information into independent channels. Each channel $h_t[i] \in \mathbb{R}^{d_{\text{state}}}$ can be interpreted as a latent risk factor. Analyse the contribution of each channel to the overall prediction:

$$\text{Risk contribution}_i = \left\| \frac{\partial \hat{r}_t}{\partial h_t[i]} \right\|_2$$

This provides a learned risk decomposition, analogous to PCA or factor models but dynamically adapted to market conditions.

10.4 Macro-Integration

Incorporate macroeconomic variables (interest rates, inflation, GDP) as auxiliary inputs:

$$x_t^{(\text{aug})} = \begin{pmatrix} x_t^{(\text{price})} \\ m_t^{(\text{macro})} \end{pmatrix}$$

Since macro data is lower frequency, use interpolation or a separate Mamba encoder before concatenation.

10.5 Integrating Eigen for Performance

The naïve matrix multiply is $O(n^3)$. The Eigen library provides:

- Cache-friendly matrix multiplication (BLAS-level optimisation)
- SIMD vectorisation
- Expression templates (zero-overhead lazy evaluation)

Migration path: Replace hand-written matrix ops with `Eigen::MatrixXd` once the forward/backward passes are verified.

10.6 Parallel Scan (Associative Scan)

In the current implementation, the selective scan is a sequential recurrence $O(T)$. The associative scan algorithm can parallelise this:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$$

This is an *associative* operation and can be computed via a parallel prefix-sum algorithm in $O(\log T)$ steps (requires GPU parallelism).

11 Phase 10: Three-Step Roadmap to Production Quant

This phase outlines a realistic path from the current single-stock CPU implementation to a production quant trading system.

11.1 Step 1: Train on Single Stock (1–2 weeks)

Goal: Verify that the Mamba can learn predictive patterns in financial time series.

- Complete all missing functions: `forward_sequence`, `backward`, `sgd_step`, `sharpe_loss`.
- Use daily OHLCV data for a single liquid stock (e.g., SPY or AAPL, 10+ years).
- Feature set: 5 lags of returns + RSI(14) + Bollinger(20,2) + volume ratio.
- Hyperparameters: $d_{\text{model}} = 64$, $d_{\text{state}} = 16$, $d_{\text{conv}} = 3$, sequence length $T = 63$ (one quarter).
- Loss: composite with $\lambda_1 = 1$ (MSE) and $\lambda_2 = 0.5$ (Sharpe).
- Monitor loss decrease, validation Sharpe, and directional accuracy.
- Run numerical gradient checking to validate all parameter gradients.

Success criterion: Validation Sharpe > 0.5 (annualised) and directional accuracy $> 55\%$ on out-of-sample data.

11.2 Step 2: Multi-Stock with Walk-Forward (1–2 months)

Goal: Scale to multi-stock prediction with robust walk-forward validation.

Key additions:

- Implement a `MultiStockMamba` class that accepts N stocks and predicts N returns simultaneously:

$$\hat{\mathbf{r}}_t = W_{\text{out}} y_t, \quad W_{\text{out}} \in \mathbb{R}^{N \times d_{\text{model}}}$$

- Add **RMSNorm** after each Mamba layer:

$$\text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{n} \sum_i x_i^2 + \epsilon}} \cdot \gamma$$

- Add **residual connections**: $y_t^{(l)} = y_t^{(l-1)} + \text{Mamba}^{(l)}(y_t^{(l-1)})$.
- Implement **walk-forward cross-validation**: re-train monthly on expanding window, evaluate on next month.
- Incorporate **transaction costs** (slippage, commission) into the Sharpe loss:

$$\text{SR}_{\text{net}} = \frac{\mu(\mathbf{r}_{\text{net}})}{\sigma(\mathbf{r}_{\text{net}})}$$

where $\mathbf{r}_{\text{net}} = \mathbf{r}_{\text{gross}} - \text{cost} \cdot |\Delta w|$.

- Replace naive $O(n^3)$ matrix multiply with a hand-tuned loop-unrolled version, or bind to **OpenBLAS**.
- Increase d_{model} to 256–512, stocks to 20–100.

Performance estimate (CPU): $d_{\text{model}} = 256$, $N = 50$ stocks ≈ 100 stocks/sec inference.

Success criterion: Out-of-sample portfolio Sharpe > 1.0 (annualised) net of estimated transaction costs.

11.3 Step 3: GPU Training & Live Trading (requires hardware)

Goal: Deploy as a live quant trading signal.

Required infrastructure:

- **cuBLAS integration**: Replace matrix operations with GPU-accelerated kernels.
- **Mixed precision**: Use TF32/BF16 for matmul, FP32 for loss-critical paths.
- **Distributed training**: Data parallel (DDP) across 4–8 GPUs for universe of 500+ stocks.
- **Data pipeline**: Daily ETL from market data feeds (e.g., Polygon, Bloomberg, Quandl).
- **Backtesting engine**: Integrate with a backtester (e.g., Backtrader, Zipline, or custom) for strategy evaluation.
- **Execution system**: Connect to brokerage API (IBKR, Alpaca, etc.) for automated trading.

Realistic timeline: 6–12 months with a small quant team.

Expected results:

- Information ratio > 1.5 on a diversified equity portfolio.
- Linear-time inference ($O(T)$) enabling real-time updates on every tick.
- Streaming state updates with constant memory, independent of market history length.
- Dynamic regime adaptation: the model automatically adjusts memory (via $\bar{A}_t$) between trending and mean-reverting regimes.

Appendix: Notation Reference

Symbol	Meaning	Shape
x_t	Input feature vector at step t	$d_x \times 1$
r_t	Actual return at step t	scalar
$\hat{r}_t$	Predicted return at step t	scalar
p_t	Asset price at step t	scalar
z	Projected input	$(2 d_{\text{model}}) \times 1$
z_1	SSM branch	$d_{\text{model}} \times 1$
z_2	Gate branch	$d_{\text{model}} \times 1$
$x'_{\text{conv},t}$	Convolution output	$d_{\text{model}} \times 1$
Δ_t	Discretised step size	$d_{\text{model}} \times 1$
A	SSM diagonal matrix	$d_{\text{model}} \times 1$ (diagonal elements)
$\bar{A}_t$	Discretised A	$d_{\text{model}} \times 1$
B_t	Input matrix	$d_{\text{state}} \times 1$
$\bar{B}_t$	Discretised B	$d_{\text{model}} \times d_{\text{state}}$
C_t	Output matrix	$d_{\text{state}} \times 1$
D	Skip connection	$d_{\text{model}} \times 1$
h_t	SSM hidden state	$d_{\text{model}} \times d_{\text{state}}$
$y_{\text{ssm},t}$	SSM output	$d_{\text{model}} \times 1$
y_t	Gated output	$d_{\text{model}} \times 1$
$\hat{p}_t$	Predicted direction probabilities	3×1 (ternary)
μ	Mean of predictions over sequence	scalar
σ	Standard deviation of predictions	scalar
SR	Sharpe ratio	scalar
τ	Quantile level	scalar
L_{MSE}	Mean squared error loss	scalar
L_{SR}	Sharpe ratio loss	scalar
L_{Dir}	Directional cross-entropy loss	scalar
L_{τ}	Pinball loss at quantile τ	scalar
λ_i	Loss weighting coefficients	scalar
W_{in}	Input projection weight	$(2 d_{\text{model}}) \times d_x$
W_{conv}	Conv1d kernel	$d_{\text{model}} \times d_{\text{conv}}$
W_{Δ}	Delta projection weight	$d_{\text{model}} \times d_{\text{model}}$
W_B	B projection weight	$d_{\text{state}} \times d_{\text{model}}$
W_C	C projection weight	$d_{\text{state}} \times d_{\text{model}}$
W_{out}	Output projection weight	$d_y \times d_{\text{model}}$
σ	Sigmoid function	—
SiLU	SiLU/Swish function	—
softplus	Softplus function	—
$\odot$	Element-wise multiplication	—
$\mathbb{I}_{\text{cond}}$	Indicator function (1 if cond true)	—
η	Learning rate	scalar